



AirTime Guide

2026/02 RNA

Contents

Introduction.....	2
How it works	2
Airtime hat	2
Build your own.....	2
PCB board - self	3
PCB board – buy	4
Adding the ferrite coil	4
Make your own coil.....	4
Buy a coil	4
Adding the ferrite coil to board	5
Using an air coil.....	7
Antenna Operating Modes	8
Standard Pi install.....	9
Installing the Airtime software	17
... or buy it from us	19
Status LED Indicators.....	20
Airtime web interface	20
Widgets.....	21
Clock Widget	21
The Control Widget	22
System Statistics Widget.....	24
Broadcast Control Widget	25
Mobile Interface:	27
Watch Configuration	27
Appendix A: watches using radio signals	29
Appendix B: wall- and desk clock.....	33

Introduction

Airtime is a compact, locally deployed time-signal transmitter designed to synchronize radio-controlled watches and clocks in regions where no national long-wave time service is available (notably the southern hemisphere).

Airtime is intentionally short-range. It is **not** a broadcast transmitter and is intended for personal, laboratory, workshop, or small-office environments.

Support two modes – amplified and non-amplified. Amplified has around 6m range, non-amplified has a 10cm range.

How it works

- Runs off a Raspberry Pi. We used and tested with Zero 2W, but other should work too. There are some limitations – see the TXTempus Github repo for limitations on which PIs work.
- The Pi syncs over the Internet using NTP.
- There is a hat on the Pi (in these docs we call it the Airtime hat) – you can connect either a ferrite coil (in amplified mode) or an air-coil (non-amplified mode). The hat also contains system LEDs and a button to manually start time sync.
- The Airtime hat transmits the time signal to your watch. It uses the TXTempus open-source project (from Henner Zeller) to transmit on various frequencies and encoding. The library supports all frequencies and encodings other than BCP. The TXTempus repo can be found here: <https://github.com/hzeller/txttempus>
- There is a web interface on the Pi that allows you to control and configure the device.

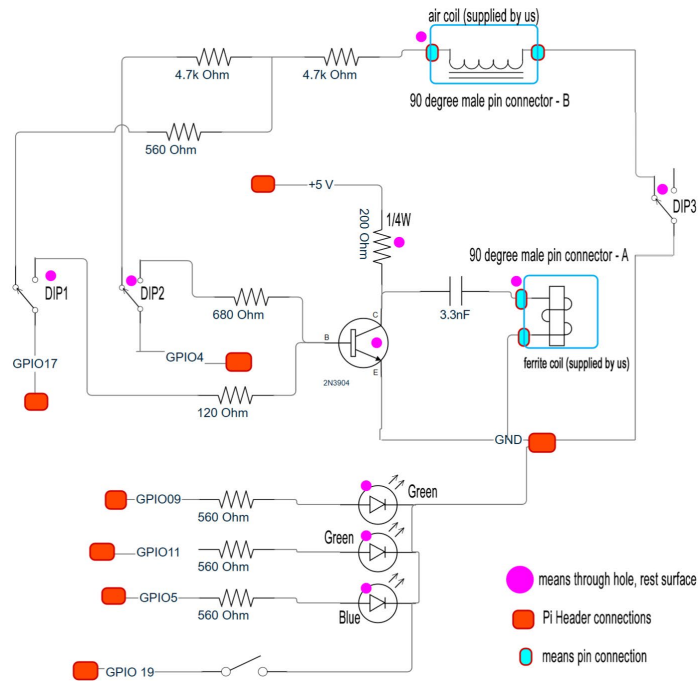
Airtime hat

The first step is to get the AirTime hat. This is basically a piece of extremely simple hardware that sits on top of the Raspberry Pi. It comes with 3 LEDs that show various statuses, a button that can be used to control the device, a simple amplifier and an antenna to transmit the signal.

Build your own

We've published our schematic on the repo (AirTime-Hat-PCB/ Schematic – brief.jpg) and you can build your own using protoboard. In testing, that's what we've done. It's ugly, but it works. Here is the circuit:



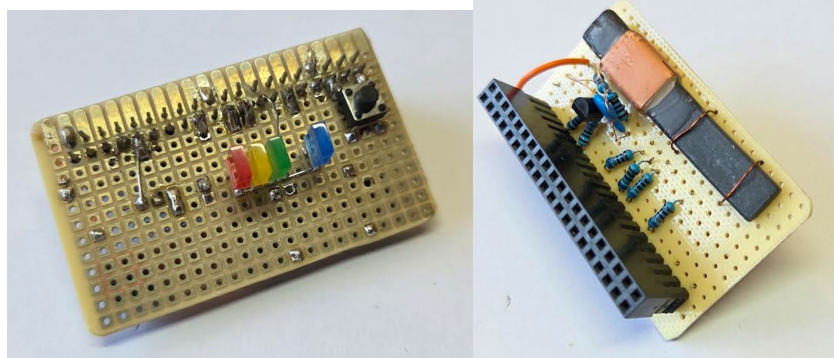


The amplifier is almost a direct copy (some values tweaked) from an amplifier that Andreas Spiess describes in depth in this YouTube video:

<https://www.youtube.com/watch?v=6SHGAEhnsYk>

The switches move the output to either the amplifier or to the aircoil (with the resistors exactly as Henner has them in his setup).

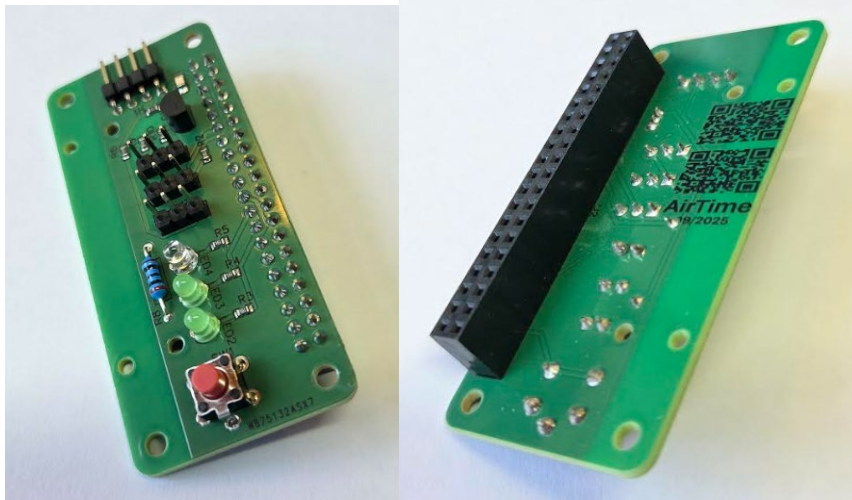
Here is the (ugly) proto board with this circuit:



PCB board- self

Of course, it's nicer to have a PCB. So, we made a PCB and sent it off to PCBWay to see it in the real world. We asked them to also populate the board with all the components except the ferrite coil.

Here's the board back from PCBWay:



Notice how there is no ferrite coil on the board.

Do it yourself: simply take the files in the `docs/AirTime-Hat-PCB/PCBWay/` folder and send them all to PCBWay. The files contain all the information for PCBWay to make these boards for you. You can decide if you want to populate the board with your own components, but we suggest letting them do it. They do it well.

PCB board– buy

You can buy the PCB from us. But we're super lazy and we hate admin and paperwork, and it means getting more boards from PCBWay and packaging it up and courier-ing it to you and it's not really our day jobs and a lot of effort. So you'll pay for that effort, but it means you don't need to deal with PCBWay (which BTW, is really easy).

It really means that you're lazier than us and that's pretty special.

Adding the ferrite coil

Do it yourself: In any case, you'll need to add the ferrite coil. We don't supply the coil at all. Why? Because customs in different countries see this as a broadcasting device and they may stop it and then there is even more drama. And we don't want drama.

Again you have options – make a coil or buy a coil.

Make your own coil

About 300 turns around a +- 10cm ferrite coil. We don't suggest you do this yourself since RF engineering is indistinguishable from magic and your force may not be strong enough.

Buy a coil

Buy a DCF77 coil and decoder. We will be using this as a receiver, but the coil and ferrite stick is nicely tuned for 77.5Khz. While this is the sweet spot, we've tested it down to 60kHz and it works OK.

You don't need the decoder, but these usually come together. This what the coil and decoder looks like:



You can find these on AliExpress or your country's local equivalent. This is not an expensive component. Here is the AliExpress link, but it will probably be non-working by the time you click on it: <https://www.aliexpress.com/item/1005007832198551.html>

Just search for 'DCF77 decoder coil'.

Adding the ferrite coil to board

The board is designed so you can add either a ferrite coil or an air coil. You select which to use by setting the jumper and connecting the antenna to the right pins.

Output

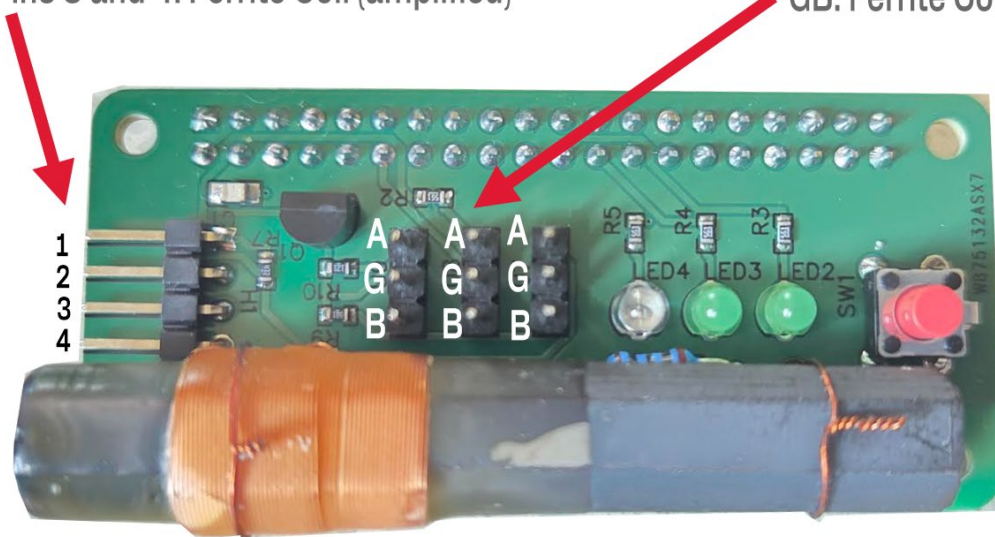
Pins 1 and 2: Air Coil (no amplification)

Pins 3 and 4: Ferrite Coil (amplified)

Jumpers:

GA: Air Coil

GB: Ferrite Coil

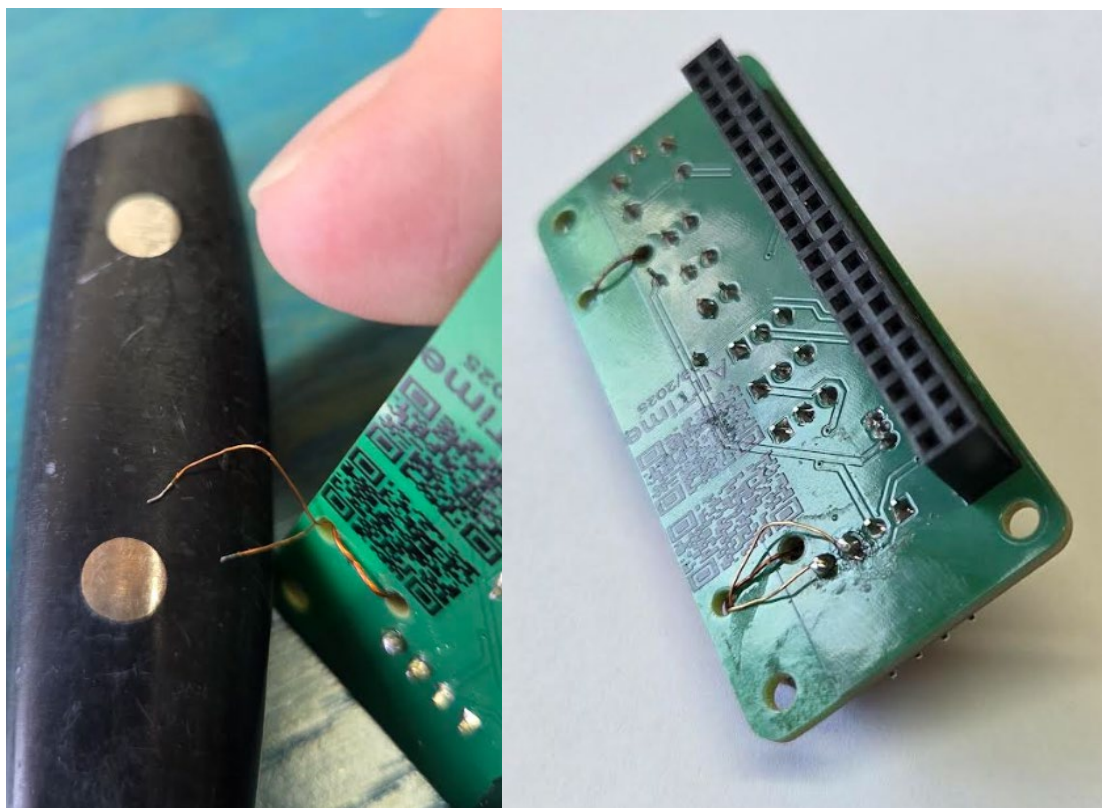


We like to route the ferrite coil's wires through some of the mounting holes and wire it directly to the pins at the bottom of the board. This way it stays in place – but you're welcome to use the pins with a connector. Just keep in mind that the antenna (both air and ferrite) should not be too far from the board (no more than 20cm).

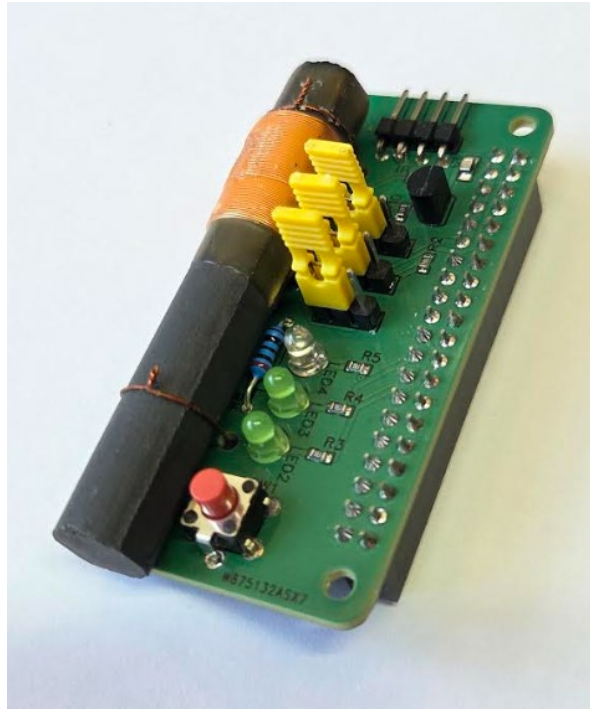
In testing we've discovered that having the ferrite coil antenna on the board give the best results, so if you have doubts, rather do that.

Place the coil in place like shown in the photo and use spare coated wire (or even threads) to secure it using the holes in the PCB board. Do not use open wire as it might short on the board below (duh).

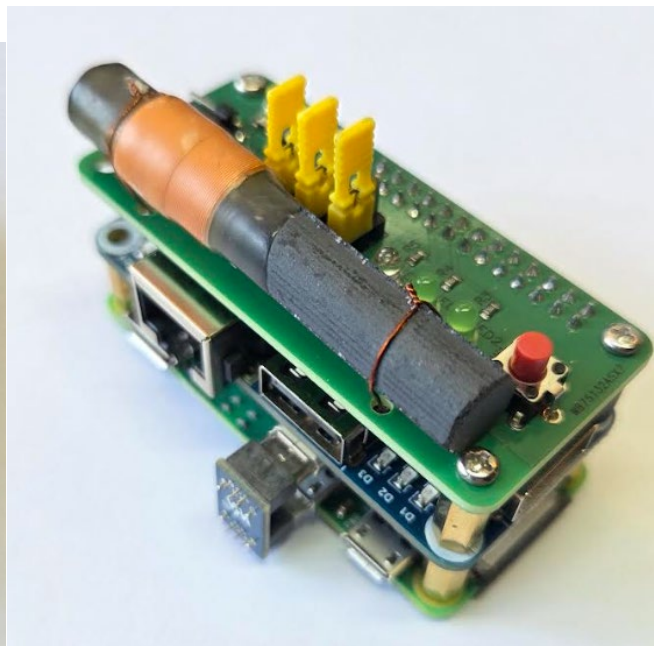
Measure how much of the coated wire you need to remove and cut it. Measure twice, cut once. Remove the coating from the wire – careful to not cut the wire obviously. We use a knife to scrape the coating off. Coat the exposed wire with solder and then solder it to pins 3 and 4:



Once the ferrite coil is nicely in place – set the jumpers. You want to connect G and B (from the picture above). With the jumpers installed (look at those yellow jumpers!) and the ferrite coil installed, it looks like this:



Woot! Your Airtime hat is complete and you can add it to your Raspberry Pi Zero 2W:



The unit on the left has no Ethernet, the unit on the right is combined with an Ethernet/USB hat between the Pi and the Airtime hat.

Using an air coil

An air coil is basically a bunch of wire loops... with air in the middle.

In experiments we've found that about 20 loops with approximately 7cm diameter works the best.

<insert picture of air coil etc>

The air coil should be connected to pins 1 and 2 and you should set all three of the jumpers to GA – see picture.

Again, don't make the wire from the board to the coil too long – 30cm max.

Antenna Operating Modes

Airtime supports **two mutually exclusive antenna modes**, selected via jumper configuration.

3.2.1 Non-Amplified Mode (Air-Coil)

Use case: Ultra-short-range synchronization with minimal emissions

- Antenna type: User-supplied air coil
- Coil specification:
 - 15–30 turns
 - Diameter: 5–10 cm
 - Transformer or enamelled copper wire
- Electrical connection: Pins **1 & 2**
- Jumper setting: **GA** (all three jumpers)

Performance:

- Typical range: up to ~10 cm
 - Orientation-sensitive
 - Ideal for regulatory-safe or bench-testing environments
-

3.2.2 Amplified Mode (Ferrite-Coil)

Use case: Room-scale synchronization

- Antenna type: Tuned ferrite rod and coil
- Electrical connection: Pins **3 & 4**
- Jumper setting: **GB** (all three jumpers)
- Includes onboard low-power amplifier

Performance:

- Typical range: up to ~5–6 m (line-of-sight)

- Strongly attenuated by walls, reinforced concrete, metal, or brick
- Field strength decreases rapidly with distance ($\sim 1/r^3$)

Regulatory note:

Users are responsible for ensuring compliance with local emission regulations. If in doubt, use non-amplified mode.

Standard Pi install

TL;DR – Get a Pi Zero 2W, install standard Pi image, give it Internet.

Do it yourself: You now need to set up the Raspberry Pi. We've asked ChatGPT to write this section. We've checked it, and it's pretty good. Of course, you need a Raspberry Pi. We use the Zero 2W.

You can get this from Amazon. Get one with pre-soldered headers (that black connector thing at the side with the pins sticking out), else you need to do this yourself and just work.

You should also consider buying spacers and heatsinks for the Pi – you get this on Amazon too. Strictly speak you don't need them, but if you want your hardware to look nice we recommend it. Lastly, you'll need a micro SD card.

Oh, and you need a MICRO USB cable. Why we don't have USB C on these Pi blows our minds, but that's how it is.

Raspberry Pi Zero 2 W – First-Time Setup Guide

Goal:

By the end of this guide, you will have:

- A Raspberry Pi Zero 2 W connected to the internet
- Its IP address
- An SSH login from your computer

What you need (before you start)

Hardware

- Raspberry Pi Zero 2 W
- microSD card (8 GB or larger)
- microSD card reader (USB is fine)
- USB power supply (5 V, ≥ 2 A)
- A Wi-Fi network with internet access



Software (free)

- **Raspberry Pi Imager**
 - Download from: <https://www.raspberrypi.com/software/>
-

Step 1: Install Raspberry Pi Imager

1. Open the Raspberry Pi website
2. Download **Raspberry Pi Imager**
3. Install it like any normal application

Nothing else is needed yet.

Step 2: Insert the microSD card

1. Put the microSD card into the USB card reader
2. Plug it into your computer

3. Ignore any pop-ups asking to format it (this will be done properly later)

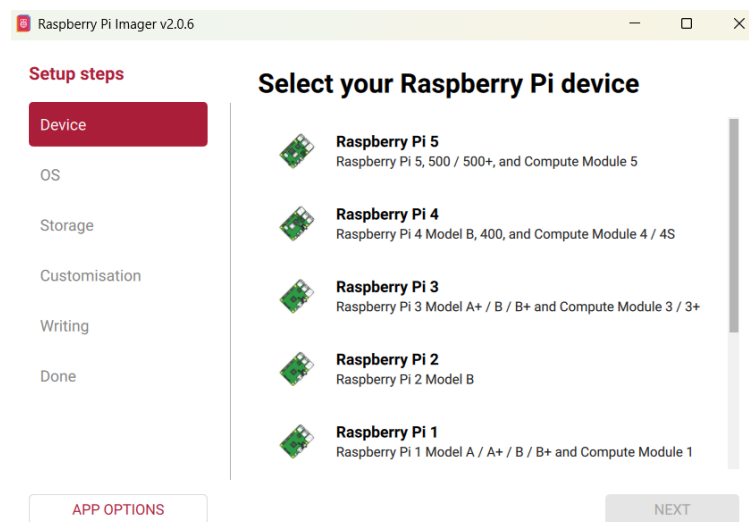
Step 3: Create the SD card image (with Wi-Fi + SSH)

This is the **most important step**.

3.1 Open Raspberry Pi Imager

You will see three sections:

- **Choose Device**
- **Choose OS**
- **Choose Storage**



3.2 Choose the device

1. Click **Choose Device**
2. Select **Raspberry Pi Zero 2 W**

3.3 Choose the operating system

1. Click **Choose OS**
2. Select:
3. Raspberry Pi OS (32-bit)

3.4 Choose storage

1. Click **Choose Storage**

2. Select your microSD card

⚠ Double-check you select the SD card, not your hard drive

3.5 Open Customization (**CRITICAL**)

In the settings window:

✓ Set hostname

Example:

Raspberrypi

✓ Set locale settings

- Timezone (this is NB!)
- Keyboard layout (not critical, but do it)

✓ Set username and password

Example:

Username: time

Password: time (or anything you're going to remember)

✓ Configure wireless LAN

- Enter your **Wi-Fi network name (SSID)** – yes you need to type this out
- Enter your **Wi-Fi password**
- Wireless LAN country: select your country

✓ Enable SSH

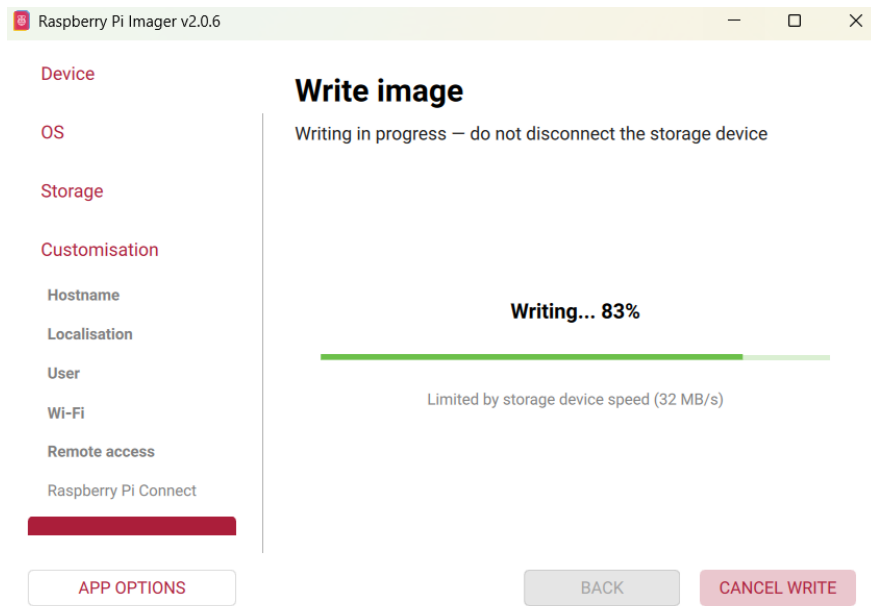
- Select **Use password authentication**

✓ Raspberry Pi Connect

- You can leave this disabled
-

3.7 Write the SD card

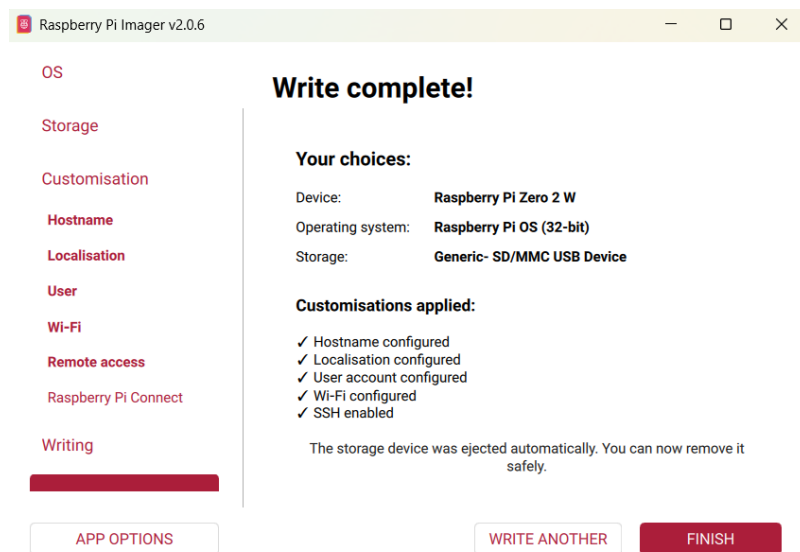
1. Click **Save**
2. Click **Yes** when asked to apply settings
3. Click **Yes** to erase the SD card



Wait until:

“Write successful”

Then close Raspberry Pi Imager.



Step 4: Boot the Raspberry Pi

1. Remove the microSD card from your computer
2. Insert it into the Raspberry Pi Zero 2 W
3. Plug in the power cable

That's it.

There is **no screen output**. The Pi boots silently.

 **Wait 2–3 minutes** for first boot.

Note: If the Airtime hat is on the Pi and the Airtime software is not installed the LEDs will NOT be on.

Step 5: Find the Pi's IP address

You now need the Pi's network address. We assume your router is giving the Pi an IP address automatically via DHCP.

Option A (recommended): Check your router

1. Log into your home router
2. Look for **DHCP Clients**, **Connected Devices**, or **LAN Status**
3. Find a device named:
4. Raspberrypi (or whatever name you gave it)
5. Note the IP address (example: 192.168.1.42)

Here is an example of a host that was called 'time5':

All (52) Wireless (38) Wired (14)				
USERNAME	IP ADDRESS	STATUS	SSID/NETWORK	AP/PORT
 time5	10.0.0.122	CONNECTED	101Net	B_AP_HQ

Option B: Use your computer (Windows)

1. Open **Command Prompt**
2. Run:

arp -a

3. Look for a new local IP address after powering the Pi

Option C: Use your computer (macOS / Linux)

1. Open **Terminal**
2. Run:

arp -a

Option D (often works): Try hostname directly

Many networks support this automatically.

You can try:

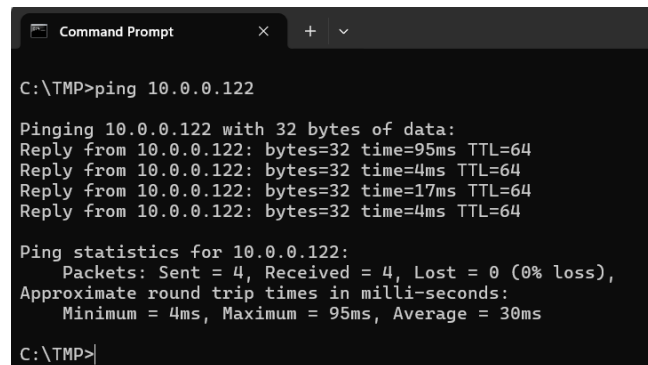
raspberrypi.local

If it works, you don't even need the IP address.

Step 6: SSH into the Raspberry Pi

SSH means “remote command-line access”. You will need this to add the AirTime software to the Pi. In theory you only need to do this once.

PS: Before you SSH into the Pi, you might want to check that you can ping the IP address – e.g.



```
Command Prompt
C:\TMP>ping 10.0.0.122

Pinging 10.0.0.122 with 32 bytes of data:
Reply from 10.0.0.122: bytes=32 time=95ms TTL=64
Reply from 10.0.0.122: bytes=32 time=4ms TTL=64
Reply from 10.0.0.122: bytes=32 time=17ms TTL=64
Reply from 10.0.0.122: bytes=32 time=4ms TTL=64

Ping statistics for 10.0.0.122:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 4ms, Maximum = 95ms, Average = 30ms

C:\TMP>
```

Windows (Windows 10/11)

1. Open **Command Prompt**
2. Run:

```
ssh time@raspberrypi.local
```

or, if hostname doesn't work:

```
ssh time@192.168.1.42
```

When asked:

Are you sure you want to continue connecting?

Type:

yes

Enter the password you set earlier.

macOS / Linux

1. Open **Terminal**
2. Run:

```
ssh time@raspberrypi.local
```

or:

ssh time@192.168.1.42

Accept the security prompt and enter your password.

Step 7: Verify internet access (sanity check)

Once logged in, type:

ping -c 3 google.com

If you see replies, your Pi is:

- Connected to Wi-Fi
- Has internet access
- Fully operational

```
C:\TMP>ssh time@10.0.0.122
The authenticity of host '10.0.0.122 (10.0.0.122)' can't be established.
ED25519 key fingerprint is SHA256:xGgo2yuijp6h7o2jkUTSIWNtUFaT+UOPrFP/Ob22v6Y.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.0.122' (ED25519) to the list of known hosts.
time@10.0.0.122's password:
Linux time5 6.12.47+rpt-rpi-v7 #1 SMP Raspbian 1:6.12.47-1+rpt1 (2025-09-16) armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
time@time5:~$ ping -c 3 google.com
PING google.com (142.251.216.14) 56(84) bytes of data:
64 bytes from lcjnbi-af-in-f14.1e100.net (142.251.216.14): icmp_seq=1 ttl=117 time=14.0 ms
64 bytes from lcjnbi-af-in-f14.1e100.net (142.251.216.14): icmp_seq=2 ttl=117 time=12.7 ms
64 bytes from lcjnbi-af-in-f14.1e100.net (142.251.216.14): icmp_seq=3 ttl=117 time=9.71 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 9.705/12.147/13.994/1.801 ms
time@time5:~$
```

You are done 🎉

At this point:

- The Raspberry Pi is online
- You can access it remotely
- No monitor or keyboard was ever required

Common problems (quick fixes)

✗ SSH connection refused

- SSH not enabled → redo SD card image and **enable SSH**

✗ Cannot find IP address

- Wait 2–3 minutes after power-up
- Check Wi-Fi credentials carefully
- Ensure 2.4 GHz Wi-Fi (Pi Zero does **not** support 5 GHz only networks)

✗ **raspberrypi.local does not work**

- Use the numeric IP instead
- Some routers disable local name resolution

Installing the Airtime software

SSH into the Pi with username 'time' and password 'time' (or whatever you've set)

Enter the following into the command line:

```
sudo git clone https://github.com/aleh11/airtime
```

The output would be this:

```
time@time5:~ $ sudo git clone https://github.com/aleh11/airtime
Cloning into 'airtime'...
remote: Enumerating objects: 103, done.
remote: Counting objects: 100% (103/103), done.
remote: Compressing objects: 100% (75/75), done.
remote: Total 103 (delta 39), reused 78 (delta 20), pack-reused 0 (from 0)
Receiving objects: 100% (103/103), 3.50 MiB | 3.01 MiB/s, done.
Resolving deltas: 100% (39/39), done.
time@time5:~ $ |
```

You now have the AirTime repo on the Pi. You may go to the airtime directory and run the install script

```
cd airtime && sudo ./install.sh
```

Follow the prompt to install the software:

```
time@time5:~ $ cd airtime && sudo ./install.sh
=====
AirTime Installation
=====

This script will perform the following actions:
• Install system packages (Python, SQLite, Nginx, Chrony, Git)
• Install Python packages (gpiozero, python-crontab)
• Configure system services (systemd units)
• Modify system configuration files
• Create and configure database directories

Do you wish to proceed with the installation? (y/n) y

Installing AirTime at: /home/time/airtime

Updating package lists...
```

The install can take some time. Be patient, this isn't a supercomputer. When the installation is complete it will look like this:

```

Starting airtime-status...
● airtime-status.service - AirTime Hardware Status Monitor (GPIO/LEDs)
   Loaded: loaded (/etc/systemd/system/airtime-status.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-01 15:54:06 SAST; 1s ago
 Invocation: f5307a2b7e1047348700d34bddbcc358
   Process: 4583 ExecStartPre=/home/time/airtime/airtime-server/gpio-cleanup.sh (code=exited, status=0/SUCCESS)
  Main PID: 4595 (python)
    Tasks: 1 (limit: 359)
       CPU: 705ms
   CGroup: /system.slice/airtime-status.service
           └─4595 python /home/time/airtime/airtime-server/backend/systemStatus.py

Feb 01 15:54:05 time5 systemd[1]: Starting airtime-status.service - AirTime Hardware Status Monitor (GPIO/LEDs)...
Feb 01 15:54:06 time5 systemd[1]: Started airtime-status.service - AirTime Hardware Status Monitor (GPIO/LEDs).

Restarting nginx...
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Drop-In: /etc/systemd/system/nginx.service.d
            └─airtime.conf
   Active: active (running) since Sun 2026-02-01 15:54:07 SAST; 1s ago
 Invocation: 612fd53028b144278d9b87422edbcc5c
    Docs: man:nginx(8)
   Process: 4602 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 4604 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Main PID: 4605 (nginx)
    Tasks: 5 (limit: 359)
       CPU: 187ms
   CGroup: /system.slice/nginx.service
           └─4605 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─4606 "nginx: worker process"
               └─4607 "nginx: worker process"
                 └─4608 "nginx: worker process"
                   └─4610 "nginx: worker process"

Feb 01 15:54:07 time5 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Feb 01 15:54:07 time5 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.

=====
Installation Complete!
=====

Service Management Commands:
  Status:  sudo systemctl status airtime-server
           sudo systemctl status airtime-status
           sudo systemctl status nginx

  Start:   sudo systemctl start airtime-server
           sudo systemctl start airtime-status
           sudo systemctl start nginx

  Stop:    sudo systemctl stop airtime-server
           sudo systemctl stop airtime-status
           sudo systemctl stop nginx

  Restart: sudo ./restart.sh

  Logs:    sudo journalctl -u airtime-server -f
           sudo journalctl -u airtime-status -f
           sudo tail -f /var/log/nginx/airtime-access.log
           sudo tail -f /var/log/nginx/airtime-error.log

Dashboard URL: http://10.0.0.122/

time@time5:~/airtime $ |

```

Woot, the Airtime software has been installed! You will immediately see the LEDs lights on the board start flashing. You can test the hardware by pressing the button the Airtime hat (hold it for ½ second) and the blue LED light should switch on.

... or buy it from us

You can buy the fully installed Pi from us. But we're super lazy and we hate admin and paperwork, and it means work and packaging it up and courier-ing it to you and it's not really our day jobs and a lot of effort. So you'll pay for that effort, but it means you don't need to do it yourself (which BTW, is really easy).

It really means that you're even lazier than us -- and that's REALLY lazy.

Status LED Indicators

6.1 LED Behaviour

- **Startup:** Sequential flashing indicates successful boot
- **Normal operation:** Both green LEDs flashing

Fault indication:

- LEDs permanently off → no power or no connectivity
- LEDs permanently on → fault condition

6.2 LED Functions

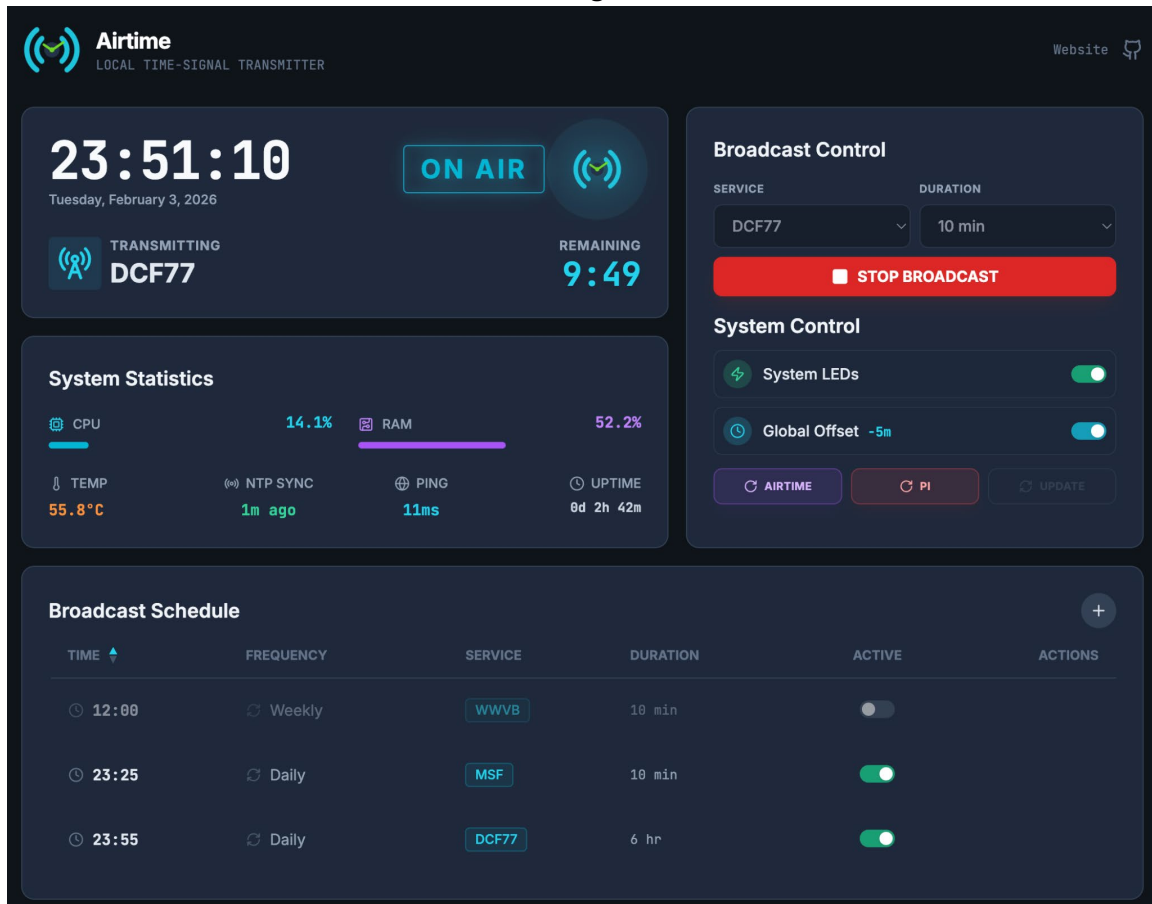
- **LED 2 (Green):** Internet reachability (ICMP to 1.1.1.1)
- **LED 3 (Green):** Last successful time synchronization
- **Blue LED:** Active time transmission

The duration of LED “on” time reflects network latency: shorter on-time indicates better connectivity and more recent platform time syncs.

Airtime web interface

Browse to the IP address of the AirTime device. You can also set a DNS name for it on your router. Ensure you are using `http://` and not `https://` since this device does not have SSL on it and you don't want to manage certificates either. There is no password on this device. The worse someone in your network can do to this device is to offset the time. Here is a picture of

what the web UI looks like at the time of making this document:

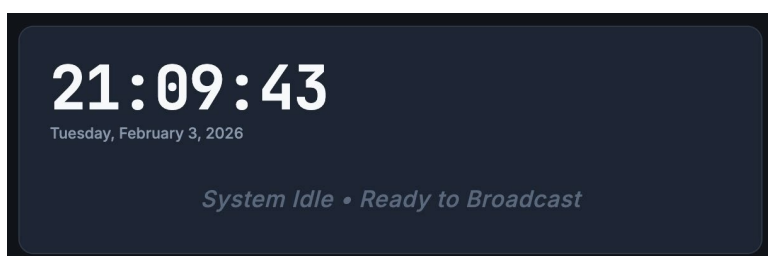


Widgets

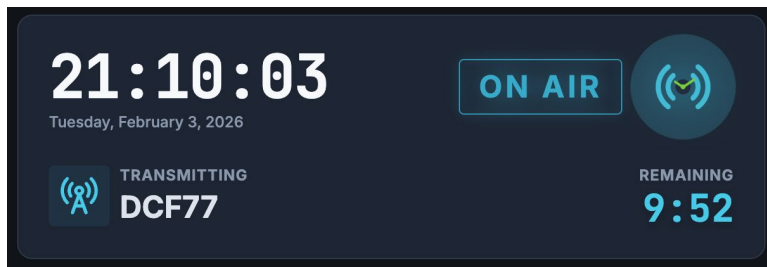
The web interface consists of 4 widgets/cards, each of which provide insight and control of the Airtime device.

Clock Widget

The clock widget has 2 main views, one when the system is on and broadcasting, and one when it is idle, as seen below.



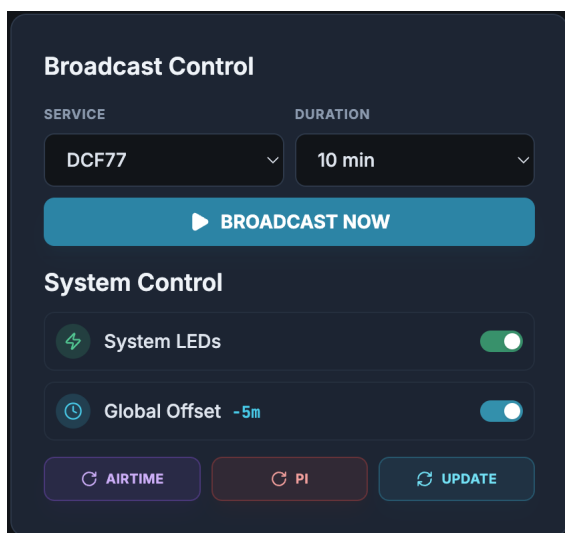
When the system is **NOT** broadcasting this widget simply displays the time and date. It is important to note that the time displayed is that of the **Airtime** device's time, not your browser running the interface – this is done intentionally so you are aware that the time your watch/clocks receive is what you expect it to be.



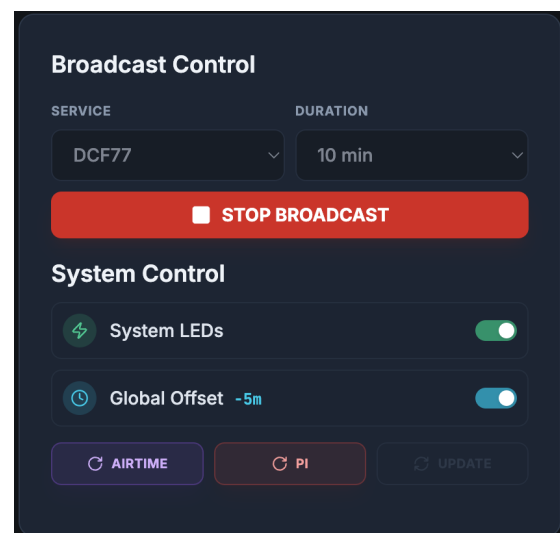
The clock widget transforms when the **Airtime** system is broadcasting a signal, with several indicators appearing. This shows the service currently transmitting (DCF77, JJY, MSF, etc.) as well as the time remaining. Additional bright UI elements (Pulsing Logo & oversized “ON AIR” block) are added so that it is super quick and easy to identify whether the antenna is running (even with your eyes squinted).

The Control Widget

This widget is where all the system configuration takes place, and as with the clock widget,



System is idle



System is broadcasting

there are 2 main states – one for broadcasting, and one for idle.

Lets go through each of the settings/controls:

Service dropdown: This allows you to choose from the 5 available services, i.e (DCF77, MSF, JJY40, JJY60 and WWVB). Your selected service will also save in between page refreshes, server updates and Pi reboots. NB: The service dropdown is disabled during broadcasts, because why are you changing stuff while your watch is trying to sync????

Duration dropdown: The duration settings are also set as a dropdown, this was done to improve UX, so you don't have a clunky number input field that you need to fill out each time you want to change the duration. The options consist of the following times : (10m,

20m, 30m ,1hr, 2hrs, 4hrs and 6hrs). For the most part, you'd want to keep this to an option with a duration of less than 1hr, as longer broadcasts should probably be set as a scheduled broadcast (more on that later). NB: As with the service dropdown, the time adjustment is also disabled while the system is broadcasting.

Broadcast button: This button is what actually starts the antenna broadcast, by running the txtempus binary, and runs with your selected service and duration that you set above. Once you click **BROADCAST NOW** the Clock widget will update accordingly, with the same configured duration and service, this will also change the button into the **STOP BROADCAST** mode, which as stated, will terminate any running broadcast, this includes broadcasts running from a schedule or button press, so BE CAREFUL not to prematurely end a 6hr schedule :)))

System LEDs toggle: This toggle allows you to turn the 3 LEDs located on the Airtime hat off/on. As covered earlier, these LEDs indicate the system's Internet reachability, time sync and current broadcast status. This option was added for users who don't necessarily want to see the LEDs, i.e have the airtime in a concealed place or don't like the blinding effect of blue LEDs in general... Additionally, all of these system statuses are also displayed within the dashboard UI (more on this later).

Global offset setting: This is superrrrr important to understand. The reason that this is called **GLOBAL** offset is because well, it is global. That means that any offset you add here will apply to a physical button press, a click of the **BROADCAST NOW** button and most importantly, any scheduled broadcasts. If you have multiple broadcast's already in place and you toggle/ configure a new **GLOBAL** offset, these existing schedules are automatically updated with the new offset. Clicking the offset toggle for the first time will bring up the config popup which looks like this:

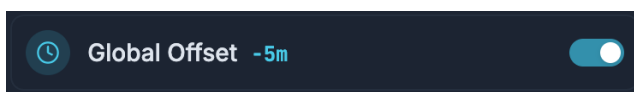
The image displays two side-by-side screenshots of the 'Global Time Offset' configuration interface. Both screens feature a dark theme with a warning message at the top: 'WARNING: Modifying this will apply the offset to EVERYTHING (Scheduled Crons, Manual Broadcasts, and UI Button events)'. Below the warning, there are two input fields for 'HOURS' and 'MINS'. In the left screenshot, the hours are set to '2' and minutes to '00'. In the right screenshot, the hours are set to '0' and minutes to '05'. Below these fields are two buttons: 'BEHIND (-)' and 'AHEAD (+)'. In the left screenshot, the 'AHEAD (+)' button is highlighted in green, while in the right screenshot, the 'BEHIND (-)' button is highlighted in orange. At the bottom of each screen is a large blue button labeled 'Apply Offset'.

We added both hour and minute offsets, and unlike the time duration dropdown, we have left this completely configurable. The offset sets 2 major purposes – the first being an easy way to check if your Airtime broadcast is successfully syncing your watch(es),

as it's super easy to see if the set time is a few minutes ahead/behind what it is supposed to be.

The second purpose, and more important, is issues relating to timezones. When experimenting with several watches and services, there exist cases in which even though the watch receives say 09h00, the service selected changes it to UTC time (this happens with WWVB) and is covered in wayyy more depth in the Watch Configuration section later in the guide. The point is, you can adjust this offset to whatever your situation is , so that the time your watch receives is always what YOU want it to be. This also means that all those people who are late to everything can set their watches 5(0) minutes early to stop annoying their friends/family.

Finally, once this offset is set, you can quickly toggle it on/off and the offset will be visually shown as below (even when it is off):



NB: as with the service/time dropdowns, the offset toggle is also disabled while the system is broadcasting, because again, why would we want to change this while a watch is syncing...

Restart/Update Buttons: The final components of the Control Widget consist of 3 beautifully coloured buttons:



- **Restart Airtime:** This restarts **ALL** of the Airtime related services (System monitor [LEDs, buttons and system stats], the Python Backend [System settings and stored DB content] and the Server hosting dashboard. This is for if something seems to be going wrong, i.e LED toggle is not working, button is not working or UI elements seem to be wrong/misleading.
- **Reboot Pi:** As the name suggests, this applies a full reboot of the Airtime Pi, this is here for if you are worried about temperatures / CPU usage / RAM usage or any other reason. This will also attempt to keep you on the page (staring at an awesome loading animation until the Pi restarts and the dashboard is started).
- **Check for update:** This button simply checks with the airtime repo if there are any new features/ stability updates, if there are not, a popup will just tell you this. In the case that updates are available, another popup will appear allowing you to “update now”, you can also just hit cancel on this and it will go away until you check for an

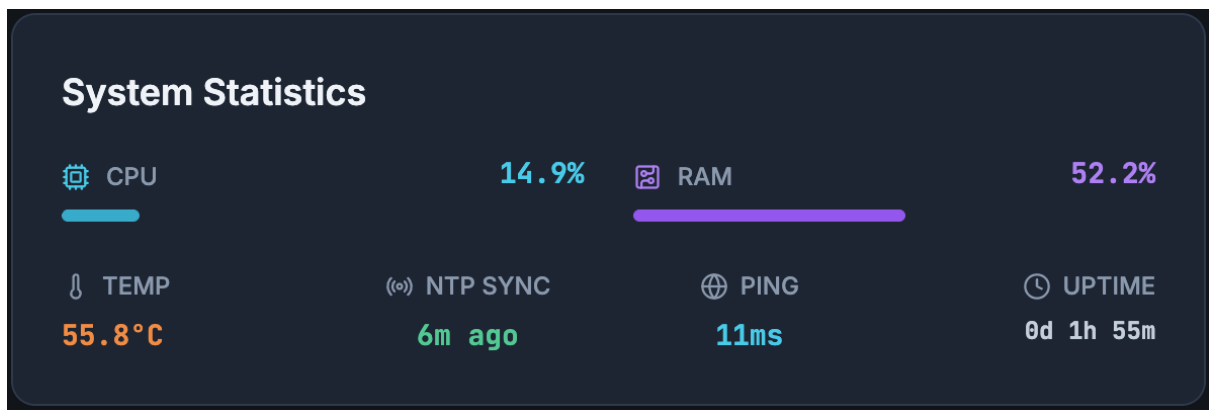


update again.

applying an update will also restart the Airtime system, to ensure that all new features are running.

System Statistics Widget

This widget allows you to see what's going on with your Airtime Pi device, providing useful insight into how it is performing. These metrics are updated every 2 seconds.



Most of these statistics/metrics are self explanatory as to why they are useful, as if the Pi is struggling with 100% CPU or memory it might be a good idea to restart it. The metrics you see in the screenshot above are the normal operating numbers, so it is a good baseline for if you are worried about something.

Generally, the RAM usage is stable at around 50% (remember this Pi only has around half a gig of RAM) so if this is touching 80/90% and you are noticing a laggy interface/actions , then it might be a good idea to restart the Pi. The same goes for the CPU, don't panic if you see it spike up every so often (this is normal) but if it is constantly at 100%, then again, think about restarting the Pi. If issues related to CPU/RAM are consistent, ssh into the pi and make sure you do not have additional scripts/services running too (ChatGPT can help you debug this).

The Pi's temperature should also not really stay about 70°C for too long, if it is in an enclosure, make sure there is proper airflow/circulation. This is also why we strongly recommend getting a heatsink for your Pi.

Lastly, the NTP sync and Ping are metrics specifically curated by us, as it's good to know that A: your device has been recently synced to a NTP time server and B: your internet is not kak (bad). Generally, you want as low ping as possible, and the NTP sync should typically not be longer than a handful of minutes ago, if you see these numbers get too high, consider rebooting your Pi.

Broadcast Control Widget

The final widget is the broadcast control widget, where you can add/remove/edit/enable/disable broadcast services from running at certain times.

TIME	FREQUENCY	SERVICE	DURATION	ACTIVE	ACTIONS
23:55	Daily	DCF77	6 hr	☒	
23:25	Daily	MSF	10 min	☒	LIVE ⚡
12:00	Weekly	WWVB	10 min	☐	

By default this table will be populated with only one schedule, **23:55 DCF77 for 6hrs** one. This is because most watches automatically sync in the early hours of the morning, and it is set to DCF77, well because that's what we use.

You are welcome to add as many schedules as you want, and each schedule can use a different service. This is useful if you have certain watches locked to a specific service and other watches locked to different services. Obviously you cannot broadcast the 2 different services at a time, so don't try this.. If you do try this, or even just have services overlapping, the SECOND service will immediately kill the current running service, so do some proper planning here to avoid this being an issue.

There are also 2 frequency options, daily and weekly – these are pretty self explanatory and the weekly option will run every Sunday. The service and duration dropdowns are the same as the ones found in the control panel, for the same reasoning as explained there.

Any scheduled broadcast can also be toggled on and off if need be, and a running service is also nicely highlighted just for fun. Also, if you have millions of broadcasts scheduled you can sort asc/desc on the time/frequency/service and duration columns.

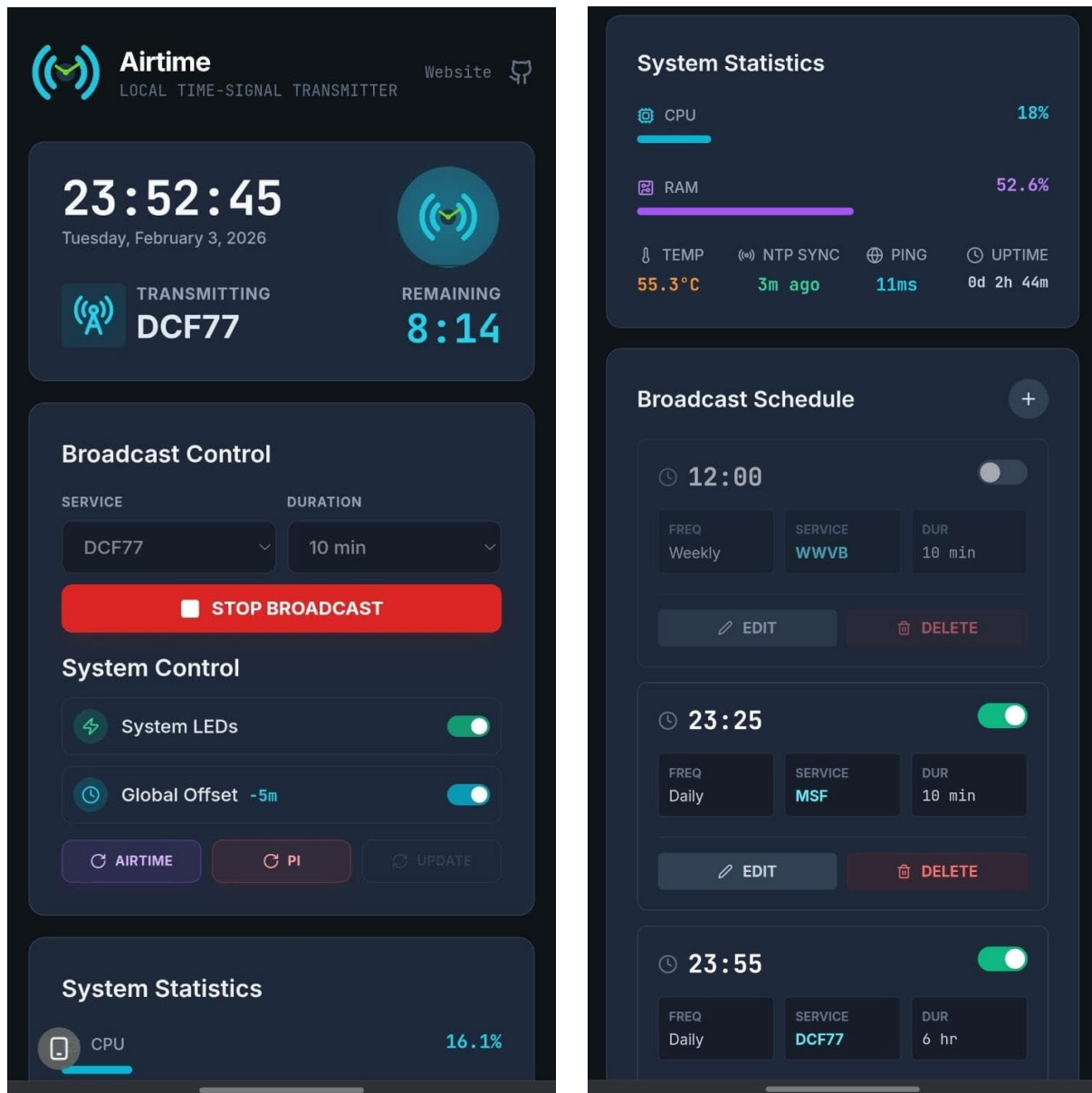
Note: this table reflects 1:1 with the sudo crontab, if you don't believe us, you can always check/edit this manually using the following command (but why would you):

```
sudo crontab -e
```

If none of that made any sense to you then that's okay, just ignore it.

Mobile Interface:

The dashboard is also optimised for viewing it on a cellphone, this is obviously not limited to android/apple devices as it is just a website (lol). All of the functionality remains the same and the cards are rendered slightly differently for a better mobile UX. Here is kinda what it looks like:



Watch Configuration

You can set your watch's time zone to be in Berlin, Germany, disable daylight saving and use the DCF77 encoding. Even if you're in Sydney, Australia, you can set your watch to this time zone and use Airtime to sync the watch to the correct time.

The watch will still think it's in Berlin, but the time is whatever the transmitter is transmitting. The only reason why you don't want to always do this is that if you look in the configuration you'll see the Home Time to be Berlin and it might bother you.

You can play around with setting the watch to different time zones, use different frequencies (and encodings) and also set the offset. You are 100% bound to find a configuration that will work for you.

Supported stations include:

- DCF77 (Germany) – 77.5 kHz: covers most of Western Europe, up to Romania.
- WWVB (USA, Fort Collins) – 60 kHz: covers North America, small parts of Northern south America.
- MSF (UK, Anthorn) – 60 kHz: Covers UK
- JJY (Japan) – 40 kHz / 60 kHz: Covers Japan and surroundings.

Note: There is one exception to this – because the American transmitter is located in Colorado and can only be transmitting a single time and America is stretched over multiple timezones, you need to be careful with doing this trick with the WWVB frequency. WWVB broadcasts the time in Coordinated Universal Time (UTC). So, if you set your timezone to be New York the watch will assuming the signal is UTC time and adjust it by 4 or 5 hours depending on summer or winter time. Of course, you can fix this by doing a +5h offset and disabling daylight saving, but this feels kind of sub-optimal 😊

It's takes between 2 minutes and 5 minutes for your watch to fully sync, so be patient. It's highly recommended that you test the distance and orientation of your setup before you assume it's working.

When you're ready to start the sync, press the button on the AirTime device. Ensure that you're using the correct frequency and encoding given the time zone you've set the watch in!

NB: For some watches (like the Casios), the watch will not even attempt to sync time if the watch is not in a time zone where a transmitter is available.

We have tried and tested the following watches with Airtime but we're pretty confident that you'll find a combination of schedule, frequency, time zone that will work for you.

Brand Model Manual Sync Auto Sync

Brand	Model	Manual Sync	Auto Sync	Comments
Casio	GW-M5610	Yes	Yes 02:00	
Casio	GW-5000u	Yes	Yes 02:00	
Casio	Oceanus	Yes	Yes	Need to set to Japan timezone to auto sync
Seiko	Astron	Yes	Yes	
Citizen	Attesa	Yes	Yes	

Appendix A: watches using radio signals

There are MANY watches that use radio signals for time synchronization. Each brand calls this something else. For instance, Casio talks about ‘Multiband 6’ while Citizen calls them ‘Radio controlled’ and German watches calls it ‘Funkuhr’.

Brand-by-brand list (not exhaustive by SKU, but covers the known product families)

CASIO (Japan)



Casio is the most prolific maker across multiple sub-brands:

- **CASIO “Wave Ceptor”** (core Casio line): marketed explicitly as Casio’s radio-controlled range.
- **G-SHOCK** (“Atomic” / “Multi-Band 6” models): many squares and tool models are Multi-Band 6 (radio-controlled).
- **EDIFICE**: a dedicated “Radio-controlled (Multi-Band 6)” collection page exists for Edifice.
- **OCEANUS**: “Multi-Band 6” technology page describes receiving standard radio waves from 6 stations.
- **PRO TREK**: portions of the Pro Trek line include radio-controlled variants (often combined with solar and sensors).

Common Casio marketing terms to search: *Wave Ceptor, Multi-Band 6, Atomic Timekeeping, Radio-controlled.*

CITIZEN (Japan)



Citizen sells radio-sync under Eco-Drive and higher-end lines:

- **Eco-Drive Radio-Controlled / Atomic Timekeeping:** Citizen explicitly markets watches that “receive standard time signals” in its Promaster Sky materials.
- **Citizen EU “Radio Controlled” category:** Citizen’s European site maintains a radio-controlled product category (examples shown include and other Eco-Drive models).

Common Citizen terms to search: *Radio Controlled, Atomic Timekeeping, Eco-Drive radio controlled, Hxxx* module references (movement families vary by market).

SEIKO (Japan)



Seiko has long produced radio-sync quartz movements (often JDM-focused):

- **Seiko radio-signal receiving watches (6 stations / 5 areas):** Seiko’s own instructions describe receiving signals from **Japan (2), China, USA, Germany, UK**.
- **Seiko calibers and lines commonly associated with radio control:**

- **7Bxx** and **8Bxx** families (example documentation for 8B92 describes a “radio-controlled watch” syncing to official standard signals).
- **BRIGHTZ** (notably many “Radio Wave Control”/solar models marketed for multi-region reception).

Common Seiko terms to search: *Radio Wave Control, Standard radio signal, 7B62/7B72, 8B92, Brightz.*

ORIENT (Japan)



Orient has produced (primarily Japan-market) solar radio-controlled models:

- **Orient “Neo70s” / WV series radio-controlled solar** (example review of **WV0071SE** as a radio-controlled solar model).
- Additional WV-series examples exist in resale listings (e.g., WV0061SE described as radio-controlled solar).

Common Orient terms: *WV**SE, radio-controlled solar, JDM.*

JUNGHANS (Germany)



One of the historically significant radio-controlled watch makers:

- **Junghans MEGA:** widely recognized as an early radio-controlled wristwatch line; the “Mega” article notes its significance in early 1990s radio-sync wristwatches.
- Modern Junghans collections include **MEGA / MEGA Solar** variants sold as radio-controlled.

Common Junghans terms: *MEGA*, *MEGA Solar*, *Funk* (German marketing often uses *Funkuhr*).

BERING (Denmark brand; many models produced with radio-sync modules)



Bering provides explicit operating instructions for radio-controlled watches:

- **Bering Radio-Controlled Watch:** their manual describes daily automatic reception and gives MSF (UK) as an example transmitter.

Common Bering terms: *Radio-controlled*, *Funk*, *MSF/DCF77* in manuals.

Appendix B: wall- and desk clock

(note this some of the wall clocks ONLY work on certain frequencies – this is because American manufactures cannot conceive that people might use their products outside of the USA).

Not notes here are some weather stations that also make use of radio-based time signals.

Atomic Wall Clocks (Radio-Controlled)

1. La Crosse Technology WT-3126B 12-Inch Atomic Wall Clock

- Analog radio-controlled wall clock receiving WWVB (USA) atomic time signal.
- Automatic synchronization and daylight-saving adjustment.

2. Bulova Atomic Analog Wall Clock Model C5003

- Classic analog wall clock with automatic time-signal reception.
- No manual setting required; updates time and DST automatically.

3. Atomic Wall Clock with Night Light

- Self-setting atomic wall clock with backlight/illumination.
- Auto DST adjustment and silent movement (no ticking noise).

4. Orium Silver Radio Controlled LED Wall Clock 11345

- LED-display radio-controlled wall clock with automatic time correction.
- Modern digital format with clear numeric time display.

Atomic/Radio-Controlled Desk/Table Clocks

5. WallarGe Atomic Wall/Desk Clock

- Dual-function clock: can stand on a desk or mount on a wall.
- Digital atomic synchronization with indoor temperature and humidity readouts.
- Self-setting and DST adjusting.

Additional Models & Categories

Beyond the above specific listings, there are broader product families and typical offerings in the atomic (radio-controlled) clock space:

Analog and Digital Wall Clocks:

- Many *La Crosse Technology* clocks (both analog and digital) automatically tune to the WWVB broadcast from NIST. [L](#)
- Large-digit scientific/industrial variants (e.g., Fisherbrand Traceable) sync to WWVB for labs, offices, and school environments.

Desk Clocks:

- Clock families from various OEMs that include **radio-controlled desk clocks** with automatic time correction are available (often with alarm/temperature functions).

Branded Specialty Options:

- *Casio “Wave Ceptor”* technology extends beyond watches; some branded wall clocks incorporate Wave Ceptor receivers for automatic time setting.